

A way back — letting the routing score recover on its own

To: Helen Achebe, Director of Product, Dispatch & Supply

From: Product Manager, Rook Dispatch · **Date:** 23 September 2026

From the point of view of the person it happens to, not a setting — per your note.

1. The story

In 2019, Wen left a comment next to `record_declined()` in `history.py` asking whether a responder's recent-acceptance score should ease back toward neutral on its own. It was left unresolved: *"leaving it as-is for now."* A decline or timeout drops the score `0.12`; an accept only recovers `0.08`; nothing brings it back by itself. That sat fine for six years — until 4.2 weighted proximity up and recent acceptance down, and a responder parked at the floor stopped being offered enough work to ever climb back off it.

Meteor Mite — Kip's other responder — went from 11 offers a week to 1 after 4.2, while The Gale, same handler, same city, same weeks, went from 13 to 21. She and three others (The Undertow, Vesper, Farlight) lost 80%+ of their weekly offers and never recovered; unaided, the odds of climbing back are roughly **1 in 2.8 million**. With a recovery path, her score eases back toward neutral on its own after a stretch with no offers — she isn't winning her way out of a hole that only deepens. Kip gets no dashboard, no dial. What he'd notice is simpler: her phone starts ringing again.



What this deliberately doesn't do:

- **Doesn't touch the 4.2 reweighting** — `WEIGHT_PROXIMITY` / `WEIGHT_RECENT_ACCEPTANCE` stay as 4.2 set them; Priya's warning against relitigating 4.2 still holds.
- **Doesn't touch the 60-second timeout** — a separate, already-understood problem.
- **Doesn't single out habitual decliners** — same formula, same treatment, for everyone.
- **Isn't a setting** — no toggle, dial, or handler-facing control.
- **Isn't the fix for the "is my account broken?" tickets** — that needs responder-visible history, not this.

2. Open questions before coding

- **Neither side would know it's working** — nothing yet surfaces the change to Meteor Mite or Kip; a good week and a lucky week would look identical.
- **"Unoffered" isn't defined precisely enough to size** — suppression vs. a genuine coverage gap.
- **No backfill decision** — does Meteor Mite's existing time at the floor count, or does the clock start at zero on ship day.
- **No success bar** — what "worked" looks like from Kip's side, and by when.
- **No monitoring or rollback plan** — 4.2 also shipped clean, no pages, and still caused undetected harm.

3. Where the code needs to change

- **history.py** — core of the change. Needs a per-responder last-updated timestamp (doesn't exist today) and new decay logic, most likely computed lazily inside `recent_acceptance()`.
- **config.py** — two new tunables alongside `ACCEPTANCE_CREDIT` / `DECLINE_PENALTY`: how long unoffered before decay starts, how big each step is.
- **offer.py** / **routing.py** — no changes expected; `offer.py` only fires history events for responders actually offered, confirming the clock must run on elapsed time, not an event; `routing.py` already reads `history.recent_acceptance()`, so it picks up decayed values automatically.
- **availability.py** — no changes expected, but any shape change to a responder record must go through Supply first, per that file's own note.
- **Migration** — a one-time decision, not a file: backfill a last-updated time for already-suppressed responders, or start every clock at zero on ship day.

Recommend a short working session with Wen, Sofia, and Nadia to close section 2 before anyone touches section 3.

Can bring a clickable walkthrough next, if that's useful alongside this.